CSES – Advanced Graph Problems Reference

All tasks: 1 s, 512 MB. Paths/walks and output witnesses are stated explicitly where relevant.

3303 – Nearest Shops

An undirected graph has anime shops in k cities. For every city, find the shortest distance to a shop in a *different* city; a local shop does not count for that city's answer. **In:** `n m k`; k shop cities; then m undirected edges
Out: n distances; -1 if no other shop is reachable
Constraints: $k \le n \le 10^5$; $m \le 2 \cdot 10^5$. Graph may be disconnected.

1134 – Prüfer Code

Decode a Prüfer code of length $n-2$ into its labeled tree. The code convention repeatedly removes the smallest-labeled current leaf and records its neighbor. **In:** `n`; then $n-2$ code values
Out: $n-1$ tree edges in any order
Constraints: $3 \le n \le 2 \cdot 10^5$; code values in $[1, n]$.

1702 – Tree Traversals

A binary tree has distinct labels. Given its preorder and inorder traversals, reconstruct and print the postorder traversal. **In:** `n`; preorder; inorder
Out: postorder traversal
Constraints: $n \le 10^5$; input is guaranteed to describe a valid binary tree.

1757 – Course Schedule II

Given prerequisite edges $a \to b$, output the valid course order that completes course 1 as early as possible, then course 2 as early as possible, and so on. This priority is by course number, not simply by the next available label.
In: `n m`; then m prerequisites `a b`
Out: n courses in the required order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$; at least one valid schedule exists.

1756 – Acyclic Graph Edges

Orient every edge of an undirected graph so that the resulting directed graph has no directed cycle. **In:** `n m`; then m undirected edges
Out: m directed edges; any valid orientation
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.

2177 – Strongly Connected Edges

Orient every edge of a simple undirected graph so that the resulting directed graph is strongly connected. **In:** `n m`; then m undirected edges
Out: a valid orientation, or `IMPOSSIBLE`
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.

2179 – Even Outdegree Edges

Orient every edge of a simple undirected graph so that every vertex has even outdegree. **In:** `n m`; then m undirected edges
Out: a valid orientation, or `IMPOSSIBLE`
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.

1707 – Graph Girth

Find the length (number of edges) of the shortest cycle in a simple undirected graph. **In:** `n m`; then m undirected edges
Out: girth, or -1 if the graph is acyclic
Constraints: $n \le 2500$; $m \le 5000$.

3357 – Fixed Length Walk Queries

For each query (a, b, x) in a connected simple undirected graph, determine whether a *walk* of exactly x edges can start at a and end at b; vertices/edges may be revisited.
In: `n m q`; edges; then queries `a b x`
Out: `YES/NO` per query
Constraints: $n \le 2500$; $m \le 5000$; $q \le 10^5$; $0 \le x \le 10^9$.

3111 – Transfer Speeds Sum

A weighted tree has edge transfer speeds. For vertices a, b, $d(a, b)$ is the minimum edge speed on their unique path. Sum $d(a, b)$ over all unordered vertex pairs. **In:** `n`; then $n-1$ edges `a b x`
Out: $\sum_{a<b} d(a, b)$
Constraints: $n \le 2 \cdot 10^5$; $x \le 10^6$. Use 64-bit.

3407 – MST Edge Check

For every edge of a connected weighted simple graph, determine whether it can belong to *some* minimum spanning tree. **In:** `n m`; then edges `a b w`
Out: `YES/NO` for each edge in input order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$; $w \le 10^9$.

3408 – MST Edge Set Check

For each queried set of edge indices, determine whether there exists one MST that contains *all* edges in that set simultaneously. **In:** `n m q`; weighted edges; then q edge-index sets
Out: `YES/NO` per set
Constraints: $n \le 10^5$; $m, q \le 2 \cdot 10^5$; total queried-set size $\le m$.

3409 – MST Edge Cost

For every edge, find the minimum possible spanning-tree cost under the condition that this edge must be included.
In: `n m`; then edges `a b w`
Out: conditioned minimum cost for each edge in input order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$; $w \le 10^9$. Use 64-bit.

1677 – Network Breakdown

An undirected network starts with m edges; k specified connections break one by one. Report the number of connected components after each deletion. **In:** `n m k`; initial edges; then k broken edges
Out: k component counts, in breakdown order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$; $1 \le k \le m$.

CSES – Advanced Graph Problems Reference

Continued. All tasks: 1 s, 512 MB.

3114 – Tree Coin Collecting I

A tree has some coin vertices. For each (a, b), find the minimum length of a walk from a to b that visits *at least one* coin vertex; detours are allowed. **In:** `n q`; coin flags; tree edges; then q pairs `a b`
Out: minimum walk length per query
Constraints: $n, q \le 2 \cdot 10^5$; at least one coin exists.

3149 – Tree Coin Collecting II

A tree has some coin vertices. For each (a, b), find the minimum length of a walk from a to b that visits *every* coin vertex. **In:** `n q`; coin flags; tree edges; then q pairs `a b`
Out: minimum walk length per query
Constraints: $n, q \le 2 \cdot 10^5$; at least one coin exists. Use 64-bit if desired.

1700 – Tree Isomorphism I

For each test, compare two rooted trees of equal size. Node 1 is the root in both; determine whether the rooted tree shapes are isomorphic (labels otherwise irrelevant). **In:** `t`; per test: `n`, then edges of tree 1 and tree 2
Out: `YES`/`NO` per test
Constraints: sum of $n \le 10^5$; $2 \le n \le 10^5$.

1701 – Tree Isomorphism II

For each test, compare two *unrooted* trees of equal size and determine whether their shapes are isomorphic; vertex labels are irrelevant. **In:** `t`; per test: `n`, then edges of tree 1 and tree 2
Out: `YES`/`NO` per test
Constraints: sum of $n \le 10^5$; $2 \le n \le 10^5$.

1699 – Flight Route Requests

Initially there are no directed flights. Given reachability requests $a \rightsquigarrow b$, find the minimum number of one-way flight connections whose transitive reachability satisfies every request. **In:** `n m`; then m distinct requested pairs `a b`
Out: minimum number of flights needed
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.

1703 – Critical Cities

In a directed graph with a route from 1 to n, find all cities that occur on *every* route from 1 to n. **In:** `n m`; then m directed edges
Out: count, then critical cities in increasing order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.

1203 – Visiting Cities

In a directed positive-weight graph, consider only minimum-price routes from city 1 to city n. Find all cities that occur on *every shortest-price route*. **In:** `n m`; then flights `a b c`
Out: count, then mandatory cities in increasing order
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$; $c \le 10^9$; a route $1 \to n$ exists.

3308 – Graph Coloring

Color a simple graph using the minimum possible number of colors so adjacent vertices differ, and output an optimal coloring. **In:** `n m`; then m undirected edges
Out: chromatic number k, then n colors in $[1, k]$
Constraints: $n \le 16$; $m \le n(n-1)/2$.

3158 – Bus Companies

Company j sells a ticket of cost c_j that lets you travel directly between any two cities it serves. Starting from city 1, find the cheapest route cost to every city. **In:** `n m`; costs c_j; for each company, its number/list of served cities
Out: n minimum costs from city 1
Constraints: $n, m \le 10^5$; total company-city incidences $\le 2 \cdot 10^5$; $c_j \le 10^9$.

3358 – Split into Two Paths

Given a DAG, partition all vertices into exactly two vertex-disjoint directed paths so every vertex appears in exactly one path. One path may be empty; unused graph edges do not matter. **In:** `n m`; then m directed edges
Out: `YES` and the two path vertex lists, or `NO`
Constraints: $n \le 2 \cdot 10^5$; $m \le 5 \cdot 10^5$.

1704 – Network Renovation

A connected network is initially a tree. Add the minimum number of new undirected connections so the network remains connected after the failure of *any one* connection.
In: `n`; then $n-1$ tree edges
Out: minimum k, then k added edges; any optimum solution
Constraints: $3 \le n \le 10^5$.

1705 – Forbidden Cities

A connected undirected graph is fixed. For each query (a, b, c), determine whether there is a route from a to b that never visits forbidden city c. **In:** `n m q`; graph edges; then queries `a b c`
Out: `YES`/`NO` per query
Constraints: $n, q \le 10^5$; $m \le 2 \cdot 10^5$.

1752 – Creating Offices

In a tree, choose as many office vertices as possible so the distance between every pair of chosen vertices is at least d. **In:** `n d`; then $n-1$ tree edges
Out: maximum count, then any optimal set of cities
Constraints: $n, d \le 2 \cdot 10^5$.

1685 – New Flight Routes

Add the minimum number of directed flights to a directed graph so every city becomes reachable from every other city. **In:** `n m`; then m directed edges
Out: minimum k, then k added flights; any optimum solution
Constraints: $n \le 10^5$; $m \le 2 \cdot 10^5$.